

Module 9 Week A — Integration Task: SPARQL

Query Suite

Due: Thursday end of day. Resubmissions are accepted through Saturday of the assignment week.

The integration task takes you from the three verification queries in the lab to a real SPARQL query suite. You are given a richer ontology — a publications graph (~120 authors, ~80 papers, ~40 venues, ~25 topics, ~1,265 triples total) — already loaded into Fuseki. Your job is to author **eight queries** that exercise every SPARQL grammar form and clause you read about: `SELECT`, `CONSTRUCT`, `ASK`, `WHERE`, `FILTER`, `OPTIONAL`, `ORDER BY`, `LIMIT`.

You are not modeling this ontology — it is provided in full. You are *querying* it. This is consolidation work: take the patterns from the reading and apply them at scale.

Setup

Submission flow — all Module 9 Week A assignments. Module 9 Week A uses a fork-and-submit flow (drill, lab, integration) instead of GitHub Classroom. GitHub is sunsetting Classroom (full transition 2026-08-28). Steps 1–3 below are the fork-and-submit part; the rest are the same workflow you used in the lab.

If you haven't already installed the M9 packages, go back to the [Setup: install before the concept session](#) section of the reading and complete the install before proceeding.

- Fork the template repo to your GitHub account.** Open github.com/LevelUp-Applied-AI/m9-i9a and click **Fork** (top right) → set the owner to your personal GitHub account → click **Create fork**. Leave "Copy the main branch only" checked.
- Enable Actions on your fork.** On your forked repo, click the **Actions** tab. GitHub disables workflows on forks by default — click **I understand my workflows, go ahead and enable them**. (One-time per fork. The autograder cannot run on your PR until this is done.)
- Clone your fork (not the upstream repo):**

```
git clone https://github.com/<your-github-username>/m9-i9a.git
cd m9-i9a
```

- Install dependencies (cached from the lab — should be fast):** `pip install -r requirements.txt`
- Bring up Fuseki:** `docker compose up -d`. Wait until `curl -s http://localhost:3030/$/ping` returns a timestamp.
- Load the publications dataset:** `python load_dataset.py` — already complete in the starter; POSTs `data/publications.ttl` into the `/publications` dataset. Verify with `curl -G http://localhost:3030/publications/sparql --data-urlencode "query=ASK { ?s ?p ?o }" -H "Accept: application/json"` — you should see `true`.
- Create your working branch:** `git checkout -b integration-sparql-queries`

The dataset — publications

`data/publications.ttl` models a small academic graph:

Class	Approximate count
<code>:Author</code> (with <code>rdfs:label</code> and <code>skos:prefLabel</code> / <code>skos:altLabel</code> for name variants)	~120
<code>:Paper</code> (with <code>rdfs:label</code> , <code>:year</code> literal, <code>:citationCount</code> integer literal)	~80
<code>:Venue</code> (with <code>rdfs:label</code> , e.g., <code>!NeurIPS</code> , <code>!ACL</code>)	~40
<code>:Topic</code> (with <code>rdfs:label</code>)	~25

The relations:

Predicate	Subject → Object
<code>:authoredBy</code>	<code>:Paper</code> → <code>:Author</code>
<code>:publishedIn</code>	<code>:Paper</code> → <code>:Venue</code>
<code>:topic</code>	<code>:Paper</code> → <code>:Topic</code>
<code>:doi</code>	<code>:Paper</code> → string literal (DOI — Digital Object Identifier; OPTIONAL — most but not all papers have one)
<code>:citationCount</code>	<code>:Paper</code> → integer literal (every paper has one)

Citation representation (load-bearing — read carefully). Each paper carries its citation count as a single **integer literal** on the `:citationCount` predicate. There are **no `:cites` edges** in this dataset. The "top 5 most-cited papers" query orders by that literal — it does not aggregate over inverse citation edges.

Your task

Complete `queries.py`. Eight functions, each returning a SPARQL string.

```
# queries.py — your task is to fill in the SPARQL string each function returns

def q1():
    """
    Q1 — List all authors who have published at venue :NeurIPS.

    Variables in the SELECT: ?author.
    Pattern: ?paper :authoredBy ?author ; :publishedIn :NeurIPS .
    """
    return "" # TODO

def q2():
    """
    Q2 — For each topic, count the number of papers on that topic.

    Variables in the SELECT: ?topic ?n.
    Use GROUP BY ?topic and COUNT(?paper) AS ?n.
    """
    return "" # TODO

def q3():
    """
    Q3 — List all author-coauthor pairs in CANONICAL FORM — one row per
    unordered pair (?a, ?b), never both (a,b) and (b,a). Two distinct
    authors are coauthors if they share at least one paper.

    Variables in the SELECT: ?a ?b.

    Two requirements — omit either and the row count is wrong:
    1. SELECT DISTINCT ?a ?b
       — coauthors who share multiple papers otherwise produce one row
       per shared paper. DISTINCT collapses them to one row per pair.
    2. FILTER (str(?a) < str(?b))
       — makes the pair canonical (lexicographically ordered by URI
       string). Without it, each unordered pair appears twice.
    """
    return "" # TODO

def q4():
    """
    Q4 — List every paper and its DOI, with the DOI OPTIONAL. Papers
    without a :doi triple must still appear in the results (with ?doi
    unbound). Use OPTIONAL { ... }.

    Variables in the SELECT: ?paper ?doi.
    The mandatory pattern is ?paper a :Paper . — the OPTIONAL wraps the
    :doi triple. Putting the :doi triple in the main WHERE block instead
    of inside OPTIONAL drops every paper without a DOI; the autograder
    catches that.
    """
    return "" # TODO

def q5():
    """
    Q5 — Return TRUE if any author has more than 10 papers; otherwise FALSE.

    Use ASK. Match a single author binding and count their papers via a
    sub-SELECT with HAVING (COUNT(?p) > 10).
    """
    return "" # TODO

def q6():
    """
    Q6 — Return a graph (CONSTRUCT, not SELECT) of all papers published
    in 2023, with each paper linked to its authors. Emit triples of the
    form ?paper :authoredBy ?author.

    Hint: CONSTRUCT { ?paper :authoredBy ?author . } WHERE { ... ?paper :year 2023
    The autograder verifies the result is a CONSTRUCT graph, not a row
    set — using SELECT and reshaping manually fails the test.
    """
    return "" # TODO

def q7():
    """
    Q7 — Return the top 5 most-cited papers ordered by descending citation
    count, using the literal :citationCount property.

    Variables in the SELECT: ?paper ?cc.
    Pattern: ?paper :citationCount ?cc.
    Clauses: ORDER BY DESC(?cc) LIMIT 5.

    Note: there are NO :cites edges in this dataset. Attempting to COUNT
    inverse-citation edges will return zero rows; the autograder rejects
    that approach.
    """
    return "" # TODO

def q8():
    """
    Q8 — List all authors whose name matches "Hinton" via either
    skos:prefLabel OR skos:altLabel (the SKOS disambiguation pattern from
    reading §7).

    Variables in the SELECT: ?author.
    Use FILTER (!label = skos:prefLabel || ?label = skos:altLabel)
    with ?author ?label "Hinton".
    Matching only on skos:prefLabel misses any author whose canonical
    name is something else but has "Hinton" as an altLabel — the
    autograder catches that.
    """
    return "" # TODO
```

`learner_notes.md`

Alongside `queries.py`, write a brief `learner_notes.md` with **one sentence of intent** and a **small result snapshot per query**. The intent statement says *what the query answers in business terms*; the snapshot is the first 5 rows (or the triple count / boolean, as appropriate). A reviewer should be able to read `learner_notes.md` and understand the suite without running it.

Template:

```
## Q1 — Authors at NeurIPS

**Intent:** Find every author who has at least one paper published at NeurIPS.

**Result:** 14 authors.
- :hinton
- :bengio
- :lecun
- ...
```

How to run locally

```
pytest tests/ -v
```

The test file walks each function, executes the query against the live Fuseki endpoint, and checks the result against a gold baseline.

Submission

- Commit and push your branch to *your fork*:

```
git add queries.py learner_notes.md
git commit -m "Integration 9A: SPARQL query suite"
git push origin integration-sparql-queries
```

- Open a Pull Request *within your fork*** — base branch `main`, compare branch `integration-sparql-queries`. **Important:** the PR must stay inside your fork. When you click "Compare & pull request", GitHub defaults to "base repository: LevelUp-Applied-AI/m9-i9a" — change the **base repository** dropdown to your own fork (`<your-github-username>/m9-i9a`) before opening the PR. The autograder runs against your branch; check the CI status on the PR page to confirm green. If you see "Workflows are disabled" on the Actions tab, you missed Setup step 2 — enable them and push an empty commit (`git commit --allow-empty -m "ci: trigger"`) to re-run.
- Your PR description must include the rendered `learner_notes.md` (or a link to it), plus:
 - Paste your PR URL into TalentLMS → Module 9 → Integration 9A to submit this assignment.**

Challenge Extensions

Challenge Tier 1 — Set difference

Add `q9` and `q10` using `FILTER NOT EXISTS` and `MINUS`. Pick two questions that are naturally negative ("authors who have **never** published at NeurIPS"; "papers without a topic assignment"). Document one place where `FILTER NOT EXISTS` and `MINUS` produce different result sets — they treat unbound variables differently, and the difference is real. New concept: negation-as-failure semantics in SPARQL.

Challenge Tier 2 — Property paths

Write a property-path query for transitive coauthors: `?a :coauthor+ ?b`. Treat `:coauthor` as a derived relation — define it via a `CONSTRUCT` that emits `:coauthor` triples for any two authors sharing a paper, then use the `+` property path to find any pair connected by a chain of coauthorship. Document what property paths do and when they are the right tool. New concept: SPARQL property paths as Kleene-star traversal.

Challenge Tier 3 — Intent-to-query CLI

Build `query.py` — a small command-line tool that takes a fixed set of natural-language intents (e.g., `python query.py "list authors at NeurIPS"`, `python query.py "papers per topic"`) and dispatches to the matching SPARQL query against the publications dataset. Document the intent-mapping table in `CHALLENGE_TIER3.md`. New concept: an intent-to-query dispatcher as a minimal NL→formal-query reduction — it is a tiny preview of the Week B integrated pipeline.

